

Development Approach

Initially, I provided Claude with the complete set of project requirements and explained the overall expectations and objectives of the application.

After finalizing and understanding the requirements, I used Cursor to develop the application in a **phase-wise manner**. Each phase was implemented by providing Cursor with a dedicated and detailed prompt, ensuring that the development process remained structured, modular, and aligned with the defined requirements.

I will attach the prompts used for all **8 development phases** for reference.

Project: C# Console Employee Management System (training project — no LINQ allowed anywhere)

PHASE 1 OF 8 — Domain Model & Core Types

Do NOT implement collections/repositories, file I/O, delegates, threading, or the full exception-handling layer yet — those come in later phases. Build only what's listed below, but as REAL functioning features of the app, not standalone demo methods.

CONTEXT

This is a learning exercise for a trainee. The goal is to implement each concept because it's the natural solution to a real Employee Management System need — not as an isolated "look, here's concept X" method. Add inline comments explaining WHY each concept was the right tool for that specific piece of functionality. Avoid building any method whose only purpose is to showcase a language feature — the one exception is called out explicitly below.

WHAT TO BUILD

1. Project structure:

- /Models
- /Enums

(leave /Services, /Repositories, /Exceptions, /Utils for later phases)

2. Enums (in /Enums):

- `EmployeeRole`` (Intern, Developer, TeamLead, Manager, HR)
- `EmploymentStatus`` (Active, OnLeave, Terminated, Resigned)

3. `struct Address`` (in /Models) — Street, City, State, ZipCode.

Real use case: build a feature `Employee CloneWithUpdatedAddress(Employee emp, Address newAddress)`` (or similar), used when HR wants to log a "pending address change" before confirming it — copy the employee's current address into a temporary variable, modify the copy, and show that the employee's actual on-file address is untouched until explicitly saved. Comment explaining this is possible precisely because `Address`` is a value type.

4. `class Employee`` (in /Models):

- Id, Name, Role (enum), Status (enum), `Address HomeAddress``
- `DateTime? TerminationDate`` — nullable because most employees are active and simply have no termination date. Real use case: a `GetEmploymentSummary()`` method that reads naturally depending on whether this is null (e.g. "Active since..." vs "Terminated on...").
- `dynamic RoleSpecificInfo`` — real use case: this holds different data depending on Role (a Developer's info might include TechStack, a Manager's might include TeamSize).

Comment explaining `dynamic` is used here because the shape genuinely varies by role, and noting that `Dictionary<string, object>` would be a more idiomatic real-world alternative — `dynamic` is used here specifically to satisfy the requirement doc's ask.

- `static int TotalEmployeesCreated` — real use case: powers an actual `GetHeadcount()` method/property that a dashboard feature will call later, not just printed for demo purposes.

- Constructor that sets fields and increments the static counter.

- Overridden `ToString()` for real, usable employee summary output (used wherever an employee needs to be displayed).

5. `class Department` (in `/Models`):

- `Id`, `Name`, `static int TotalDepartmentsCreated`.

- Real use case: build `AssignEmployeeToDepartment(Employee emp)` that increments a real per-department employee count instance field (`int CurrentHeadcount`) — this gives static (org-wide dept count) and instance (per-department headcount) members both a genuine job to do.

6. Boxing/unboxing — EXCEPTION TO THE "REAL USE CASE ONLY" RULE.

There isn't a natural EMS feature that needs this. Put a small, clearly-labeled method (e.g. `Utils/ConceptNotes.cs` → `BoxingUnboxingExample()`) with a comment stating plainly: "This method exists solely to demonstrate boxing/unboxing as required by the project spec — there's no organic use case for it in this application's real functionality." Call it once from `Main` under a clearly labeled console header so it's honest about what it is, rather than pretending it's a feature.

7. `var` — use naturally throughout wherever the type is obvious from the right-hand side. No dedicated demonstration needed.

8. In `Program.cs` Main`, wire up a simple sequence that exercises the REAL features above (not demo prints):

- Create a couple of employees with different roles (showing `RoleSpecificInfo` differing by role)
- Assign them to a department (showing headcount incrementing)
- Call `CloneWithUpdatedAddress`` on one and print before/after to confirm the original wasn't mutated
- Call `GetEmploymentSummary()`` for an active employee and a terminated one, to show the nullable field's real branching behavior
- Print `Employee.TotalEmployeesCreated`` and a department's `CurrentHeadcount`` as real, meaningful output — not a labeled "demo" line
- Run the one honestly-labeled boxing/unboxing concept method

CONSTRAINTS

- No LINQ anywhere
- No collections/repository layer yet (a couple of local variables for this phase's demo sequence is fine)
- No file I/O yet
- No custom exceptions yet
- Single compilable console app I can run and see meaningful, feature-driven output — not a list of demo prints

OUTPUT

After writing the code, give me a short mapping of: which concept → which real feature it lives in, plus a note on the one method that's a deliberate concept-only exception (boxing/unboxing) and why.

PHASE 2 OF 8 — Object-Oriented design layer

This phase is about giving the domain model real behavioral structure — how different kinds of employees relate to each other and to shared capabilities. You decide the actual role hierarchy, what varies between roles, and how deep the hierarchy should go — whatever a thoughtful engineer would judge appropriate for a real (if small) EMS. Don't just mirror Phase 1's data with empty subclasses — there should be genuine behavioral differences that justify the hierarchy existing.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- Inheritance — a real "is-a" hierarchy where a base type captures what's common and subtypes add or specialize real behavior
- Encapsulation — genuine data protection/validation on at least one property, guarding against a real invalid state (not just private-fields-for-the-sake-of-it)
- Polymorphism via overriding — at least one method whose behavior genuinely differs per subtype, called through a base-type reference so the polymorphism is doing real work, not just present in the syntax
- Method overloading — a case where the same operation genuinely makes sense with a different set of inputs (not two overloads that do arbitrarily different things)
- Abstraction — your choice of abstract class and/or interface, used where it fits the "what something IS" vs "what something CAN DO" distinction. If you use both, briefly justify in a comment why each concept called for the tool you picked.
- Partial class — applied where a single class has genuinely grown large/multi-responsibility enough to benefit from being split (if nothing in your Phase 1+2 design organically reaches that point, use it on whichever class comes closest, and say honestly in a comment that this is a lighter-weight case for it)

DESIGN EXPECTATIONS

- Same standard as Phase 1: no method that exists purely to showcase a concept in isolation. Everything should read as a real design decision for a real EMS.
- Keep it consistent with your Phase 1 model — don't contradict decisions you already made there
- Comments should explain real engineering tradeoffs at the decision points (why this needed to be abstract vs interface, why this needed overriding vs a new method, etc.) — not line-by-line narration
- Should compile and run standalone, with Program.cs demonstrating this phase's real behavior (e.g. handling different employee types through a common reference and getting genuinely different, correct results) as an integrated part of the app's actual flow — not a separate labeled demo block

CONSTRAINTS

- No LINQ anywhere in the project
- No collections/repository layer, file I/O, threading, or custom exception classes yet (a basic guard exception for encapsulation validation is fine if needed)
- Don't build ahead into later phases' scope

OUTPUT

Explain your design reasoning after the code: what hierarchy/structure you chose and why, where each required concept ended up, and any tradeoffs — same as Phase 1.

PHASE 3 OF 8 — Collections & in-memory repository

Right now Main hires people into individual variables. That doesn't scale past a handful of hires and isn't how a real system would manage a workforce. This phase is about properly storing and managing employees as a real, queryable roster in memory.

You decide the actual repository design — what it's called, what operations it exposes, how it's structured. Think about what a real HR system would actually need to do with a roster day to day, and let that drive the design rather than mechanically wrapping each collection type in a method.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- A generic collection (e.g. `List<T>`, `Dictionary<TKey,TValue>`) used where its ordering/lookup semantics are actually the right fit for the operation
- A non-generic collection (`ArrayList` and/or `Hashtable`) — since these are legacy and arguably the "wrong" tool by modern standards, use it somewhere you can honestly justify (even if the honest justification is "the spec requires demonstrating this, and here's the tradeoff vs the generic equivalent, which I'd use in real code")
- Real, working operations on the roster that map to actual HR tasks — not just CRUD for its own sake. Think about what genuinely differs: looking someone up fast, listing people by department, listing active vs exited staff, etc. — and pick a collection type per operation based on what that access pattern actually needs (e.g. a fast id lookup vs an ordered listing want different structures)

DESIGN EXPECTATIONS

- No LINQ anywhere — any filtering/searching is done with plain loops
- Same standard as before: no method whose only job is to showcase a collection type. Everything should be there because it's the right tool for a real roster operation.
- If a collection choice is a genuine tradeoff (e.g. Dictionary for O(1) id lookup vs a List you'd scan), say so briefly in a comment — real engineering reasoning, not narration
- Keep this consistent with your Phase 1/2 model — the repository should work with the `Employee` hierarchy and department structure you already built, not a parallel simplified version

- Should compile and run standalone; Main should demonstrate the roster being used for real (hiring several people into it, looking someone up, listing a department's staff, listing exits) as part of one coherent flow

CONSTRAINTS

- No LINQ anywhere in the project
- No file I/O, custom exceptions, threading, or menu yet
- Don't build ahead into later phases' scope
- Also add the boxing/unboxing concept from the original spec, which Phase 1 skipped — a small, honestly-labeled concept method is fine since it has no organic home in this app; a one-line comment explaining that is enough

OUTPUT

Explain your design reasoning after the code: which collection you picked for which operation and why, and any tradeoffs — same as before.

PHASE 4 OF 8 — File persistence

Right now the roster lives only in memory — it's gone the moment the program exits. This phase is about making the roster durable: save it to disk, and load it back on the next run. You decide the actual file format, the save/load design, and how it fits into the repository you built in Phase 3.

Think about what a real system would actually need here — not just "write some text to a file" but a genuinely usable save/load flow: what happens on first run when no file exists yet, what happens if the file is malformed or partially written, how the polymorphic Employee hierarchy (Developer/Manager/Intern/Contractor) gets represented and correctly reconstructed on load without losing which concrete type each record was.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- File I/O for both writing and reading (System.IO) — plain text/CSV/custom delimited format is fine; you don't need JSON/XML libraries unless you think that's genuinely the better design
- `IDisposable`, used somewhere it's the correct tool — i.e. something in your file-handling code holds an unmanaged/disposable resource and needs deterministic cleanup. If your design naturally uses `StreamWriter` / `StreamReader` inside `using` blocks, that alone may satisfy this — but also consider whether a class you own (e.g. a file-backed repository or writer wrapper) should itself implement `IDisposable` because it owns a resource across multiple calls, not just one method
- A finalizer — genuinely justified only if something is holding an unmanaged resource as a safety net in case `Dispose` isn't called. If nothing in your real design needs one, say so honestly in a comment rather than forcing one in artificially — this is one of those concepts that often doesn't have a natural home in simple file I/O with `using` blocks

DESIGN EXPECTATIONS

- No LINQ anywhere
- Reconstruct the correct concrete Employee subtype on load (a saved Manager should load back as a Manager, not a generic Employee) — think about how you'll encode "which type is this row" in the file format
- Handle the realistic cases: no file yet on first run, empty file, a malformed line — decide sensible behavior for each without full custom exception handling yet (that's Phase 6); basic try/catch with a console message is fine for now
- No method that exists purely to showcase file I/O in isolation — the save/load should be genuinely wired into how the app starts up and shuts down (or however you decide is the right trigger point)
- Should compile and run standalone; Main should demonstrate a real save-then-load cycle as part of the app's actual flow (e.g. hire some people, save, and — to prove it round-trips correctly — load into a fresh repository instance and show the data, including subtype-specific fields, came back intact)

CONSTRAINTS

- No LINQ anywhere in the project
- No delegates/events, custom exception classes, threading, or menu yet
- Don't build ahead into later phases' scope

OUTPUT

Explain your design reasoning after the code: file format chosen and why, how subtype reconstruction works, where IDisposable landed and why, and whether a finalizer had a genuine use case here or was honestly omitted.

Build on top of your Phase 1–4 code — don't restart, extend/refactor as needed. Still do NOT implement custom exception classes, threading, or the menu UI — those are later phases.

PHASE 5 OF 8 — Delegates, events, lambdas, and tuples

Right now the app performs actions (hire, resign, raise, department assignment, save/load) as direct method calls with no way for other parts of the system to react to them. This phase is about introducing a real reactive/notification layer using delegates — where "something happened" and "something else responds to it" are decoupled.

You decide what actually deserves a delegate/event in this system and what the responding logic should look like. Think about what a real people-ops system would want to be notified about, and design accordingly — don't bolt a delegate onto something that doesn't need one just to hit the requirement.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- A delegate type, used to decouple an action from its response (e.g. something in the roster/repository triggers, and one or more independent handlers react — without the triggering code knowing or caring what the handlers do)
- A multicast delegate — a genuine case where more than one independent piece of logic should react to the same event (not two handlers that do arbitrarily unrelated things just to prove multicast works)
- An anonymous method or lambda, used as a handler somewhere it's actually cleaner than a named method
- A tuple, used somewhere a method genuinely needs to return more than one related value without justifying a whole new class for it (a lightweight, real "this operation naturally produces two-plus results" case)
- `virtual` / `override`` — if Phase 2 didn't already cover a case cleanly, find a real spot here where a base behavior has a sensible default that a subtype can optionally customize (this differs from Phase 2's abstract methods, which had no default — virtual is for when a default DOES make sense but subtypes may still want to change it)

DESIGN EXPECTATIONS

- No LINQ anywhere
- Events/delegates should react to real state changes already in your system (hiring, resignation, raises, department moves, save/load — your call on which ones genuinely warrant notification)
- No handler that exists purely to prove a delegate fired — each handler should do something a real system would want (e.g. logging to console in a structured way, updating a running stat, flagging something for review) — full file-based error logging is Phase 6, so keep this lightweight
- No method whose only job is to showcase delegates/tuples in isolation
- Should compile and run standalone; Main should demonstrate this phase's behavior as part of the app's real flow — hire/resign/raise a few people and show multiple independent reactions firing naturally, not a separate labeled demo block

CONSTRAINTS

- No LINQ anywhere in the project
- No custom exception classes, threading, or menu yet
- Don't build ahead into later phases' scope

OUTPUT

Explain your design reasoning after the code: which actions you wired notifications to and why those and not others, where the multicast case came from, where virtual/override landed (or why it didn't have a genuine spot), and where the tuple use case came from.

PHASE 6 OF 6 OF THE PRE-MENU CORE (Phase 6 of 8 overall) — Custom exceptions & error logging

Right now, invalid situations (negative salary, raise after exit, intern stipend cap, malformed file rows, etc.) are handled with generic exceptions and console messages, if at all. This phase is about giving the system real, meaningful error types and a durable record of what went wrong.

You decide the actual exception hierarchy and logging design. Think about what genuinely distinct failure categories already exist in your system from Phases 1–5, and design exception types around real failure modes — not one generic `EmsException`` used everywhere, and not a separate exception type invented just to have more types.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- At least one custom exception class (deriving from `Exception`` or a more specific built-in base where appropriate), used for a genuinely distinct failure condition already present in

your system — go back through Phases 1–5 and find the validation/guard points that deserve a real, named exception instead of a generic one

- If you find more than one genuinely distinct failure category (likely, given how many validation rules already exist — salary floor, intern cap, post-exit changes, malformed persisted data, etc.), consider a small custom exception hierarchy (a base app exception with specific subtypes) rather than one flat type — your call on whether that's warranted
- Structured exception handling: real `try/catch/finally`` usage at the boundaries where failures should be caught and handled gracefully (not swallowed silently, and not left to crash the app)
- File-based error logging — failures get written to a log file with enough context to be useful (what happened, when, relevant details) — decide the format and where in the app this hooks in

DESIGN EXPECTATIONS

- No LINQ anywhere
- Go back and replace the ad-hoc `ArgumentException`` /basic try-catch spots from earlier phases with your real custom exceptions where it's genuinely an improvement — this phase should tighten up validation across the app, not just add one new isolated example
- No exception type or catch block that exists purely to showcase exception handling — everything should map to a real failure mode this system can actually encounter
- Logging should be genuinely wired into where failures happen (e.g. via your Phase 5 delegate/event layer if that's a natural fit, or directly in catch blocks — your call on which is cleaner) rather than a separate "log something" demo call
- Should compile and run standalone; Main should demonstrate a few real failures occurring naturally (invalid salary, raise after exit, intern cap breach, a bad persisted record on load) and show they're caught, handled without crashing, and logged — as part of the app's real flow

CONSTRAINTS

- No LINQ anywhere in the project
- No threading, no menu yet
- Don't build ahead into later phases' scope

OUTPUT

Explain your design reasoning after the code: what exception hierarchy you chose and why, which existing validation points got upgraded to it, where logging hooks in, and any failure case you deliberately decided NOT to turn into a custom exception (and why a generic/built-in one was the better call there).

PHASE 7 OF 8 — Threading & async

Build on top of Phases 1–6. Don't implement the menu UI yet — that's Phase 8, the final phase.

This phase is about introducing real concurrency where the app can genuinely benefit from it, not concurrency for its own sake. Think about what in this system actually has latency or benefits from running independently of the main flow — file I/O (Phase 4), error logging (Phase 6), and notifications (Phase 5) are the likely candidates, but you decide what's genuinely worth making concurrent versus what should stay synchronous because making it async would add complexity without real benefit.

CONCEPTS THAT MUST APPEAR SOMEWHERE IN THIS PHASE (your call on exactly where/how):

- Multi-threading — a real background operation running on its own thread (e.g. something periodic or long-running that shouldn't block the main flow)
- `Thread.Sleep`` / `Thread.Join``, used where they're genuinely appropriate (e.g. a background thread pacing itself, or the main thread needing to wait for a background operation to finish before proceeding to something that depends on it)
- Task-based async (`async`` / `await``, `Task``), used for an operation that's naturally I/O-bound (file save/load and log writing are strong candidates — real async there is a genuine

improvement over the synchronous version from Phase 4/6, not just a wrapper for its own sake)

- Thread pooling — via `Task.Run`` or similar, where offloading work to the pool is the right call versus spinning up a dedicated `Thread``
- A cancellation token — wired into a real scenario where an in-progress operation should be stoppable (e.g. a background/periodic task that the app can signal to stop cleanly on shutdown)

DESIGN EXPECTATIONS

- Decide honestly which existing synchronous operations (file save/load, logging) are worth converting to async, and which threading/pooling use is worth adding as new functionality (e.g. a background "active headcount monitor" or periodic autosave) — don't force every concept into the same feature if that's not the cleanest design
- Should compile and run standalone; Main should demonstrate this phase's concurrency doing real work as part of the app's actual flow — e.g. an async save that doesn't block, a background task running while other work happens, and a clean cancellation on shutdown
- Be careful with thread-safety around the in-memory roster (Phase 3) and static counters (Phase 1) if background threads touch them concurrently — decide whether that's a real risk in your design and handle it appropriately (or explain why it isn't a risk given how you structured the access)

OUTPUT

Explain your design reasoning after the code: what you chose to make concurrent and why, what you deliberately kept synchronous and why, where cancellation hooks in, and how you handled (or ruled out) thread-safety concerns around shared state.

PHASE 8 OF 8 — Menu-driven console (final phase)

Build on top of Phases 1–7. This is the final phase — everything built so far (domain model, OOP hierarchy, repository, file persistence, delegates/notifications, custom exceptions/logging, threading/async) needs to come together into something a real user can actually run and interact with at runtime.

This phase is the interaction layer only — you shouldn't need to add new domain logic here, just expose what already exists through a clean console UI. If you find yourself writing significant new business logic to make the menu work, that's a signal something should have been in an earlier phase — handle it, but keep it minimal and note it in your design explanation.

WHAT THIS PHASE NEEDS TO COVER

- A real menu loop the user can navigate repeatedly until they choose to exit (not a single pass through options)
- Menu options covering the core real workflows the app already supports — your call on the exact list, but it should include at least: hiring a new employee (by type), viewing/listing the roster (e.g. by department, by status), giving a raise, assigning to a department, previewing/confirming an address move, resigning an employee, running payroll (polymorphic bonus calc), and viewing recent error log entries
- Real input validation at every point the user types something — numbers that aren't numbers, out-of-range menu choices, empty strings where a name is required, etc. should all be caught and give the user a clear message and another chance, not crash the app or silently misbehave
- Wire in the async/threading pieces from Phase 7 sensibly — e.g. save-on-exit using your async save, a background task started at launch and cancelled cleanly on exit, without making the UI feel unresponsive or confusing to use
- On startup, load the persisted roster (Phase 4) if it exists, so the app picks up where the last run left off
- On exit, ensure the roster is saved and any background work is shut down cleanly (cancellation token signaled, threads/tasks properly awaited/joined — no abrupt process kill while work is in flight)

DESIGN EXPECTATIONS

- This should feel like a coherent, usable application when run — a real person should be able to sit down and manage an employee roster through this menu without confusion
- Keep the menu code itself thin — it should mostly be calling into the repository/service methods you already built across Phases 1–7, not duplicating logic
- Handle the custom exceptions from Phase 6 gracefully at the menu level too — a validation failure (e.g. negative salary, intern cap breach) should show the user a clear message and return to the menu, not crash or produce a raw stack trace
- Should compile and run as a complete, standalone application — this is the final deliverable

OUTPUT

After writing the code, give me:

1. Your design reasoning — how you structured the menu layer and why, and any judgment calls you made about what belonged here versus what was already covered
2. A short "how to use it" walkthrough — the menu options and what each does, so I can actually run through the app myself and verify everything from Phases 1–7 is reachable and working end to end